KINGA Claims Manager Portal — Realignment and Redesign Document

Document Classification: Internal Engineering Blueprint
Portal Under Review: Claims Manager Portal (`/insurer/claims-manager`)
Document Version: 1.0 — Master Implementation Blueprint
Prepared by: KINGA Platform Engineering
Date: June 2026
Status: Approved for Implementation
Supersedes: Audit v1.0, v2.0, v3.0, Architecture Review v4.0

Purpose and Scope

This document is the single authoritative blueprint for the realignment and redesign of the KINGA Claims Manager Portal. It consolidates all findings from the four preceding audit documents and the product owner architectural review into a complete, implementable specification. Every issue identified across the audit cycle is addressed here with a precise technical specification, a backend procedure contract, a frontend component specification, and a phase assignment.

This document is structured to be read by an engineer who will implement it. It does not repeat audit findings or justify decisions — those are documented in the preceding audit series. It specifies what to build, how to build it, and in what order.

The design philosophy is stated once and applies throughout: **the Claims Manager Portal is a morning briefing, not a dashboard.** Every component answers a specific operational question. The Claims Manager should be able to answer all six core operational questions within ten seconds of opening the portal, before interacting with any tab or filter.

Part 1 — Issues Register

All issues identified across the audit cycle are consolidated here. Each issue has a unique identifier, a category, a severity, and a phase assignment. The phase assignments are defined in Part 4.

1.1 Production Defects

ID	Issue	Current Behaviour	Correct Behaviour	Phase
D-01	`closeForProcessing` calls wrong procedure	Calls `trpc.claims.approveClaim` — creates `claim_approved` audit entry; transitions to `repair_assigned`	Must call a dedicated `closeForProcessing` procedure; transitions `payment_authorized → closed` ; creates `claim_closed` audit entry	Phase 1
D-02	“Escalate” button misroutes via send-back	Calls `handleSendBack` → opens send-back dialog → calls `sendBackClaim`	Must call a dedicated `escalateClaim` procedure; transitions to `disputed` or `manual_review` ; notifies Risk Manager	Phase 1

1.2 Missing Operational Command Centre Features

ID	Feature	Current State	Target State	Phase
F-01	Queue Health Matrix	Absent — `workflowStateCounts` fetched but not rendered	Row 1: 6-stage matrix with Count, Avg Age, Oldest Claim, SLA Breaches	Phase 2
F-02	Attention Required Queue	Absent	Row 2 left panel: single count with 7-category breakdown	Phase 2
F-03	Escalation Centre	Absent	Row 2 right panel: 6-category actionable escalation surface	Phase 2
F-04	Approval Workbench	Absent — buried in Review Queue tab	Row 3 left panel: 5-metric approval command panel	Phase 2
F-05	Capacity Forecasting	Absent	Row 3 right panel: backlog trajectory indicator	Phase 2
F-06	Claims Ageing Panel	Absent — `getProcessingTimesByStage` exists but not called	Integrated into Queue Health Matrix (avg age column)	Phase 2
F-07	Workflow Bottlenecks Panel	Absent — `getBottlenecks` exists but not called	Integrated into Queue Health Matrix (SLA breach column)	Phase 2
F-08	Fleet Approvals Sidebar Navigation	Tab exists; no sidebar entry	Sidebar nav item linking to `?tab=fleet-approvals`	Phase 2

1.3 Missing Management Intelligence Features

ID	Feature	Current State	Target State	Phase
M-01	Workforce Intelligence Section	Absent — `getUserProductivity` and `getAssessorPerformance` exist but not called	Row 4: Processor Performance + Assessor Performance + Workload Distribution panels	Phase 3
M-02	Send-back Analytics	Absent — no dedicated procedure	New `getSendBackAnalytics` procedure; rework rate panel in Workforce Intelligence	Phase 3
M-03	Recovery Watchlist	Generic KPI counts only	Replaces Recovery KPI row: 4-category actionable watchlist	Phase 3
M-04	Operational Fraud Queue	Fraud Funnel widget (strategic, not operational)	Replaces Fraud Funnel: 4-category operational fraud queue in Fraud Alerts tab	Phase 3
M-05	Report Integration	11 authorised reports; zero surfaced in portal	Per-claim report buttons in Review Queue; section report buttons in Fraud Alerts	Phase 3
M-06	Capacity Forecasting Backend	`getCapacityForecast` procedure absent	New procedure: 7-day intake, 7-day completions, backlog trajectory	Phase 2 (backend)

1.4 Refinements

ID	Refinement	Current State	Target State	Phase
R-01	KPI Cards demotion	Row 1 — primary surface	Row 6 compact strip — contextual only	Phase 2
R-02	Structured send-back reason	Free-text only	Enum `sendBackReason` field + metadata storage	Phase 3
R-03	Reopen capability	`closed → disputed` transition not exposed	"Reopen" action in Processed Claims tab	Phase 4
R-04	Audit metadata: threshold recording	Automation threshold not recorded at approval	Record `requireManagerApprovalAbove` value in `workflow_audit_trail.metadata`	Phase 4
R-05	`targetRole` validation in send-back	No validation against transition rules	Validate against `WORKFLOW_TRANSITIONS` map before submission	Phase 4
R-06	Merge Recently Closed card	Redundant section below Review Queue	Merge into Processed Claims tab	Phase 4

Part 2 — New Backend Procedure Specifications

Five new procedures are required. All are derivable from existing data with no schema changes. Each specification includes the router location, input schema, query logic, and return shape.

2.1 `claims.closeForProcessing` (replaces D-01)

Router: `server/routers.ts` — `claims` router
Access: `protectedProcedure` — roles: `claims_manager`, `executive`, `insurer_admin`
Purpose: Governs the closure of a claim from `payment_authorized` state to `closed` state. Replaces the incorrect use of `approveClaim` for this action.

Input Schema:

```
z.object({
  claimId: z.number(),
  closureReason: z.string().min(10, "Closure reason required"),
  finalApprovedAmount: z.number().optional(),
})
```

Logic:

1. Fetch claim; verify `workflowState === 'payment_authorized'`
2. Verify caller has `claims_manager`, `executive`, or `insurer_admin` role
3. Call `WorkflowEngine.transition(claimId, 'payment_authorized', 'closed', userId, userRole)`
4. Update `claims.status = 'closed'`, `claims.closedAt = now()`, `claims.closedBy = userId`
5. If `finalApprovedAmount` provided, update `claims.totalClaimAmount`

6. Create `workflow_audit_trail` entry: `action = 'claim_closed'`, `comments = closureReason`, `metadata = { finalApprovedAmount }`
7. Return `{ success: true, claimId, newState: 'closed' }`

Frontend change: Replace `trpc.claims.approveClaim.useMutation` at line 252 of `ClaimsManagerDashboard.tsx` with `trpc.claims.closeForProcessing.useMutation`. Update the dialog to capture `closureReason` (required) instead of `selectedQuoteId`.

2.2 `claims.escalateClaim` (replaces D-02)

Router: `server/routers.ts` — `claims` router
Access: `protectedProcedure` — roles: `claims_manager`, `executive`, `insurer_admin`
Purpose: Escalates a claim to `disputed` or `manual_review` state and notifies the Risk Manager. Replaces the incorrect use of `sendBackClaim` for escalation.

Input Schema:

```
z.object({
  claimId: z.number(),
  escalationReason: z.enum([
    'fraud_concern',
    'high_value_dispute',
    'policy_interpretation',
    'third_party_dispute',
    'legal_threat',
    'other'
  ]),
  escalationNotes: z.string().min(10, "Escalation notes required"),
  targetState: z.enum(['disputed', 'manual_review']).default('manual_review'),
})
```

Logic:

1. Fetch claim; verify current `workflowState` is a valid escalation source
2. Verify caller role
3. Call `WorkflowEngine.transition(claimId, currentState, targetState, userId, userRole)`
4. Create `workflow_audit_trail` entry: `action = 'claim_escalated'`, `comments = escalationNotes`, `metadata = { escalationReason, previousState: currentState }`
5. Call `notifyOwner({ title: 'Claim Escalated', content: ... })` to notify Risk Manager
6. Return `{ success: true, claimId, newState: targetState }`

Frontend change: Replace the “Escalate” button handler at line 1237 of `ClaimsManagerDashboard.tsx`. Remove the `setShowSendBackDialog(true)` call. Open a new `EscalateClaimDialog` component that captures `escalationReason` (enum select) and `escalationNotes` (required textarea). Call `trpc.claims.escalateClaim.useMutation`.

2.3 `claims.getQueueHealthMatrix`

Router: `server/routers.ts` — `claims` router
Access: `protectedProcedure` — roles: `claims_manager`, `executive`, `insurer_admin`
Purpose: Returns a complete queue health matrix for all six active workflow stages. Combines count, average age, oldest claim age, and SLA breach count in a single efficient query.

Input Schema:

```
z.object({
  slaThresholdHours: z.number().default(48),
}).optional()
```

Logic: Execute a single SQL query joining `claims` with `workflow_audit_trail` to get, per active workflow stage: claim count, average hours since entering stage, maximum hours since entering stage (oldest claim), and count of claims exceeding `slaThresholdHours`.

```
SELECT
  c.workflow_state as stage,
  COUNT(c.id) as claim_count,
  AVG(TIMESTAMPDIFF(HOUR, wat.created_at, NOW())) as avg_age_hours,
  MAX(TIMESTAMPDIFF(HOUR, wat.created_at, NOW())) as oldest_claim_hours,
  SUM(CASE WHEN TIMESTAMPDIFF(HOUR, wat.created_at, NOW()) > :threshold THEN 1 ELSE 0 END) as sla_bre
FROM claims c
INNER JOIN workflow_audit_trail wat ON (
  wat.claim_id = c.id
  AND wat.id = (
    SELECT MAX(id) FROM workflow_audit_trail
    WHERE claim_id = c.id AND new_state = c.workflow_state
  )
)
WHERE c.tenant_id = :tenantId
  AND c.workflow_state IN (
    'intake_queue','under_assessment','internal_review',
    'technical_approval','financial_decision','repair_assigned'
  )
GROUP BY c.workflow_state
```

Return Shape:

```
{
  stages: Array<{
    stage: string;
    claimCount: number;
    avgAgeHours: number;
    oldestClaimHours: number;
    slaBreaches: number;
    severity: 'normal' | 'warning' | 'critical';
  }>;
  totalActive: number;
  totalSlaBreaches: number;
  generatedAt: string;
}
```

2.4 `claims.getAttentionRequired`

Router: `server/routers.ts` — `claims` router
Access: `protectedProcedure` — roles: `claims_manager`, `executive`, `insurer_admin`
Purpose: Runs seven exception rule queries in parallel and returns a structured count object for the “Attention Required” widget.

Input Schema: None (uses tenant context)

Logic: Execute seven `COUNT` queries in parallel using `Promise.all`. Each query targets the `claims` and `workflow_audit_trail` tables.

Rule	Query Logic
SLA Breach	`COUNT` of active claims where time in current state > 48 hours
High Fraud	`COUNT` of active claims where `fraudRiskScore > 80`
High Value Pending	`COUNT` of claims at `technical_approval` or `financial_decision` where `totalClaimAmount > automationThreshold`
No Update	`COUNT` of active claims where `updatedAt < NOW() - INTERVAL 7 DAY`
Multiple Send-backs	`COUNT` of claims with > 2 backward transitions in `workflow_audit_trail`
Executive Override	`COUNT` of claims with `executiveOverride = 1` in `workflow_audit_trail` in last 30 days
Disputed	`COUNT` of claims where `workflowState = 'disputed'`

Return Shape:

```
{
  total: number;
  breakdown: {
    slaBreaches: number;
    highFraud: number;
    highValuePending: number;
    noUpdate: number;
    multipleSendBacks: number;
    executiveOverride: number;
    disputed: number;
  };
  generatedAt: string;
}
```

2.5 `claims.getApprovalWorkbenchMetrics`

Router: `server/routers.ts` — `claims` router
Access: `protectedProcedure` — roles: `claims_manager`, `executive`, `insurer_admin`
Purpose: Returns lightweight aggregate metrics for the Approval Workbench panel. Uses `COUNT` and `AVG` aggregates rather than full row fetches to avoid over-fetching.

Input Schema: None

Logic:

```
SELECT
  SUM(CASE WHEN workflow_state = 'technical_approval' THEN 1 ELSE 0 END) as technical_approval_count
  SUM(CASE WHEN workflow_state = 'financial_decision' THEN 1 ELSE 0 END) as financial_decision_count
  SUM(CASE WHEN workflow_state IN ('technical_approval','financial_decision')
      AND total_claim_amount > :threshold THEN 1 ELSE 0 END) as high_value_pending,
  MAX(TIMESTAMPDIFF(HOUR, entered_approval_at, NOW())) as oldest_approval_hours,
  AVG(TIMESTAMPDIFF(HOUR, entered_approval_at, NOW())) as avg_approval_age_hours
FROM claims
WHERE tenant_id = :tenantId
  AND workflow_state IN ('technical_approval', 'financial_decision')
```

Note: `entered_approval_at` is derived from the `workflow_audit_trail` entry where `new_state IN ('technical_approval', 'financial_decision')`.

Return Shape:

```
{
  technicalApprovalCount: number;
  financialDecisionCount: number;
  highValuePending: number;
  oldestApprovalHours: number;
  avgApprovalAgeHours: number;
  totalPendingApprovals: number;
}
```

2.6 `claims.getCapacityForecast`

Router: `server/routers.ts` — `claims` router
Access: `protectedProcedure` — roles: `claims_manager`, `executive`, `insurer_admin`
Purpose: Returns 7-day intake and completion counts with a backlog trajectory direction indicator.

Input Schema: None

Logic:

```
-- Intake: claims created in last 7 days
SELECT COUNT(*) as intake_7d FROM claims
WHERE tenant_id = :tenantId AND created_at ≥ NOW() - INTERVAL 7 DAY;

-- Completions: claims closed in last 7 days
SELECT COUNT(*) as completions_7d FROM claims
WHERE tenant_id = :tenantId AND closed_at ≥ NOW() - INTERVAL 7 DAY;

-- Current backlog: active claims
SELECT COUNT(*) as current_backlog FROM claims
WHERE tenant_id = :tenantId
AND workflow_state IN ('intake_queue','under_assessment','internal_review',
                       'technical_approval','financial_decision','repair_assigned');
```

Trajectory: `intake_7d > completions_7d * 1.1` → `growing`; `intake_7d < completions_7d * 0.9` → `shrinking`; otherwise → `stable`.

Return Shape:

```
{
  intake7d: number;
  completions7d: number;
  currentBacklog: number;
  trajectory: 'growing' | 'stable' | 'shrinking';
  trajectoryDelta: number; // intake_7d - completions_7d
}
```

2.7 `recovery.getWatchlist`

Router: `server/routers.ts` — `recovery` router
Access: `protectedProcedure` — roles: `claims_manager`, `recovery_officer`, `executive`, `insurer_admin`
Purpose: Returns four actionable recovery watchlist categories to replace the generic KPI count row.

Input Schema: None

Logic:

- **Recovery Eligible:** Claims with `status = 'closed'` and third-party involvement (`thirdPartyInvolved = 1`) where no `recovery_cases` record exists for that claim. Requires a LEFT JOIN.
- **Demand Outstanding:** `recovery_cases` where `status = 'demand_sent'` and `demandLetterSentAt < NOW() - INTERVAL 30 DAY` (no response after 30 days).
- **Deadline Approaching:** `recovery_cases` where `recoveryDeadline` is within 14 days and status is not settled/closed/archived.
- **High Value:** `recovery_cases` where `approvedSettlementAmount > threshold` (use `automationPolicy.requireManagerApprovalAbove` as threshold) and status is not settled/closed/archived.

Return Shape:

```
{
  recoveryEligible: number;
  demandOutstanding: number;
  deadlineApproaching: number;
  highValue: number;
  total: number;
}
```

2.8 `workflowAnalytics.getSendBackAnalytics`

Router: `server/routers/workflow-analytics.ts`
Access: `protectedProcedure` — roles: `claims_manager`, `executive`, `insurer_admin`
Purpose: Analyses backward transitions in `workflow_audit_trail` to identify rework rates by stage and by user.

Input Schema:

```
z.object({
  startDate: z.string().optional(),
  endDate: z.string().optional(),
})
```

Logic: A backward transition is any `workflow_audit_trail` record where `new_state` is earlier in the workflow sequence than `previous_state`. The workflow sequence order is: `intake_queue(1) → under_assessment(2) → internal_review(3) → technical_approval(4) → financial_decision(5) → repair_assigned(6) → payment_authorized(7) → closed(8)`.

```
SELECT
  wat.previous_state as from_stage,
  wat.new_state as to_stage,
  COUNT(*) as send_back_count,
  COUNT(DISTINCT wat.claim_id) as affected_claims,
  COUNT(DISTINCT wat.user_id) as sending_users
FROM workflow_audit_trail wat
INNER JOIN claims c ON wat.claim_id = c.id
WHERE c.tenant_id = :tenantId
  AND wat.action LIKE '%send_back%'
  -- OR derive from state sequence comparison
GROUP BY wat.previous_state, wat.new_state
ORDER BY send_back_count DESC
```

Return Shape:

```
{
  byStage: Array<{
    fromStage: string;
    toStage: string;
    sendBackCount: number;
    affectedClaims: number;
    sendingUsers: number;
  }>;
  byUser: Array<{
    userId: number;
    userRole: string;
    sendBackCount: number;
    claimsAffected: number;
    reworkRate: number; // sendBackCount / totalTransitions
  }>;
  totalSendBacks: number;
  overallReworkRate: number;
}
```

Part 3 — Frontend Component Specifications

3.1 Target State Layout

The revised dashboard layout has six rows. The existing tab structure (Row 5) is preserved in full — no existing tabs are removed or modified except for the targeted changes specified below.

```
ROW 1 — Queue Health Matrix
6 stages × 4 columns (Count | Avg Age | Oldest | SLA)

ROW 2 — Escalation Centre
[Attention Required (left)] [6-Category Escalation (right)]

ROW 3 — Approval Workbench + Capacity Forecasting
[Approval Workbench (left 60%)] [Capacity (right 40%)]

ROW 4 — Workforce Intelligence (collapsible)
[Processor Performance] [Assessor Performance] [Workload]

ROW 5 — Existing Tabs (preserved)
Intake | Review | Active | Fraud | Processed | Fleet

ROW 6 — Compact KPI Strip
Total | Completion Rate | Savings | Avg Cycle Days
```

3.2 Row 1 — Queue Health Matrix Component

Component name: `QueueHealthMatrix`
File: `client/src/components/QueueHealthMatrix.tsx`
Data source: `trpc.claims.getQueueHealthMatrix.useQuery({ slaThresholdHours: 48 })`

The component renders a compact table with one row per workflow stage. Each row shows the stage name, claim count (with a badge coloured by count severity), average age in hours (formatted as “Xh” or “Xd Xh”), oldest claim age, and SLA breach count (red badge if > 0, grey if 0).

Each row is clickable. Clicking a row sets the `activeTab` state to the appropriate tab and applies a workflow state filter. The stage-to-tab mapping is:

Stage	Tab	Filter Applied
`intake_queue`	`intake`	None (all intake claims shown)
`under_assessment`	`active`	`workflowState = under_assessment`
`internal_review`	`active`	`workflowState = internal_review`
`technical_approval`	`review`	`workflowState = technical_approval`
`financial_decision`	`review`	`workflowState = financial_decision`
`repair_assigned`	`active`	`workflowState = repair_assigned`

Severity colouring logic:

- `claimCount`: green (< 10), amber (10–25), red (> 25)
- `avgAgeHours`: green (< 24h), amber (24–48h), red (> 48h)
- `slaBreaches`: grey (0), red (> 0)

Loading state: Render a skeleton table with 6 rows and 4 columns.

3.3 Row 2 — Attention Required Widget

Component name: `AttentionRequiredWidget`
File: `client/src/components/AttentionRequiredWidget.tsx`
Data source: `trpc.claims.getAttentionRequired.useQuery()`

The component renders as a card occupying the left half of Row 2. The card header shows “ATTENTION REQUIRED” with the total count as a large badge. The card body shows a vertical list of the seven exception categories, each with a count and a right-arrow link. Categories with count = 0 are shown in muted text. Categories with count > 0 are shown in amber or red depending on severity.

Clicking any category navigates to the Active Claims tab with the appropriate filter applied. The filter mapping is:

Category	Tab	Filter
SLA Breaches	`active`	`slaBreached = true`
High Fraud	`fraud`	`fraudRiskScore > 80`
High Value Pending	`review`	`highValue = true`
No Update	`active`	`noUpdateDays = 7`
Multiple Send-backs	`active`	`multipleSendBacks = true`
Executive Override	`active`	`executiveOverride = true`
Disputed	`active`	`workflowState = disputed`

Refresh interval: 5 minutes (`refetchInterval: 300000`).

3.4 Row 2 — Escalation Centre

Component name: `EscalationCentre`
File: `client/src/components/EscalationCentre.tsx`
Data sources:

- `trpc.claims.getEscalations.useQuery()` — disputed and high-fraud claims
- `trpc.claims.getAttentionRequired.useQuery()` — for High Value Pending, Stuck, SLA Breach counts
- `trpc.claims.getDashboardStats.useQuery()` — for executive override count

The component renders as a card occupying the right half of Row 2. It shows six escalation categories as a 2×3 grid of compact metric tiles. Each tile shows the category name, count, and a coloured indicator. Clicking a tile opens the relevant filtered view.

Tile	Count Source	Click Action
High Value Pending	`getAttentionRequired.breakdown.highValuePending`	Review Queue filtered by high value
High Fraud Risk	`getEscalations` filtered by `fraudRiskLevel`	Fraud Alerts tab
Disputed	`getEscalations` filtered by `workflowState = disputed`	Active Claims filtered by disputed
Stuck Claims	`getAttentionRequired.breakdown.noUpdate`	Active Claims filtered by no update
SLA Breaches	`getAttentionRequired.breakdown.slaBreaches`	Active Claims filtered by SLA breach
Executive Overrides	`getAttentionRequired.breakdown.executiveOverride`	Active Claims filtered by exec override

---

3.5 Row 3 — Approval Workbench

Component name: `ApprovalWorkbench`
File: `client/src/components/ApprovalWorkbench.tsx`
Data source: `trpc.claims.getApprovalWorkbenchMetrics.useQuery()`

The component renders as a card occupying the left 60% of Row 3. It shows five metrics in a 2-column layout:

- **Awaiting Technical Approval** — count with amber badge; click → Review Queue filtered to `technical_approval`
- **Awaiting Financial Decision** — count with amber badge; click → Review Queue filtered to `financial_decision`
- **High Value Pending** — count with red badge if > 0; click → Review Queue filtered by high value
- **Oldest Approval** — formatted as “Xd Xh”; red if > 48h
- **Average Approval Age** — formatted as “Xh”; amber if > 24h

A “Go to Review Queue” button at the bottom of the card navigates to the Review Queue tab.

3.6 Row 3 — Capacity Forecasting

Component name: `CapacityForecast`
File: `client/src/components/CapacityForecast.tsx`
Data source: `trpc.claims.getCapacityForecast.useQuery()`

The component renders as a card occupying the right 40% of Row 3. It shows four metrics in a 2×2 grid:

- **Current Backlog** — count
- **7-Day Intake** — count
- **7-Day Completions** — count
- **Trajectory** — directional indicator: ↑ Growing (red), → Stable (amber), ↓ Shrinking (green)

Below the grid, a single sentence summarises the situation: “Intake is outpacing completions by X claims this week — backlog is growing.” or equivalent.

3.7 Row 4 — Workforce Intelligence

Component name: `WorkforceIntelligence`
File: `client/src/components/WorkforceIntelligence.tsx`
Data sources:

- `trpc.workflowAnalytics.getUserProductivity.useQuery()`
- `trpc.analytics.getAssessorPerformance.useQuery()`
- `trpc.workflowAnalytics.getSendBackAnalytics.useQuery()` (Phase 3)

The component renders as a collapsible section with a "Workforce Intelligence" header and a chevron toggle. When expanded, it shows three panels in a 3-column grid:

Panel 1 — Processor Performance: Table of processors (role = `processor`) with columns: Name, Claims Handled (7d), Transition Count, Rework Rate (from `getSendBackAnalytics`). Sorted by Claims Handled descending.

Panel 2 — Assessor Performance: Table of assessors with columns: Name, Total Assessments, Accuracy Score, Avg Completion Time, Performance Tier. Data from `getAssessorPerformance`. Sorted by Performance Score descending.

Panel 3 — Workload Distribution: Two compact bar charts. Chart 1: Claims per processor (horizontal bar). Chart 2: Assessments per assessor (horizontal bar). Highlights overloaded users (top 20% by volume) in amber.

Default state: Collapsed. Expands on click. State persisted in `localStorage` so the user's preference is remembered.

3.8 Row 5 — Tab Structure Changes

The existing tab structure is preserved. The following targeted changes are made within existing tabs:

Review Queue Tab — Report Buttons: Each claim card in the Review Queue receives a report dropdown button (using the existing `KingaReportButton` component pattern). The dropdown offers: Claim Assessment Report (`claim.assessment`), Claim Audit Trail (`claim.audit_trail`), Cost Comparison Report (`claim.cost_comparison`).

Fraud Alerts Tab — Operational Fraud Queue: The Fraud Alerts tab header area receives a compact 4-tile summary showing: Fraud Cases Awaiting Action, Fraud Cases Awaiting Risk Review, Fraud Cases Delaying Approval, Fraud Cases Older Than SLA. These are derived from the existing `getFraudAlerts` data with state-based filtering. The existing fraud claim list below is preserved.

Fraud Alerts Tab — Escalate Button Fix: The "Escalate" button handler is replaced. It now opens a new `EscalateClaimDialog` component (see Section 3.9) instead of the send-back dialog.

Processed Claims Tab — Reopen Action (Phase 4): A "Reopen" action is added to each claim card in the Processed Claims tab. It calls a new `trpc.claims.reopenClaim` mutation that transitions `closed → disputed` with a mandatory reason.

3.9 New Dialog Components

`EscalateClaimDialog` (`client/src/components/EscalateClaimDialog.tsx`): A modal dialog with: claim number display (read-only), escalation reason (enum select — 6 options), escalation notes (required textarea, min 10 chars), target state (radio: `manual_review` default, `disputed`). Submit calls `trpc.claims.escalateClaim.useMutation`. On success: close dialog, invalidate `getFraudAlerts` and `getAttentionRequired` queries, show success toast.

`CloseForProcessingDialog` (replaces current close dialog): Update the existing close dialog to: remove `selectedQuoteId` field, add `closureReason` required textarea (min 10 chars), add optional `finalApprovedAmount` number field. Submit calls `trpc.claims.closeForProcessing.useMutation`.

3.10 Row 6 — Compact KPI Strip

The existing four KPI cards (Total Claims, Active Claims, Completed Claims, Fraud Alerts) are replaced by a compact single-row strip below the tabs. Each metric is displayed as a small inline label-value pair separated by dividers. The strip uses `getManagerOverview.kpis` data which is already fetched. No new data fetching required.

3.11 Sidebar Navigation Addition

In `DashboardLayout.tsx` or the Claims Manager sidebar configuration, add a "Fleet Approvals" navigation item:

```
{ label: "Fleet Approvals", href: "/insurer/claims-manager?tab=fleet-approvals", icon: TruckIcon }
```

The `ClaimsManagerDashboard.tsx` already reads `?tab=` from the URL query string to set `activeTab`. No additional routing logic required.

3.12 Recovery Watchlist

Component name: `RecoveryWatchlist`
File: `client/src/components/RecoveryWatchlist.tsx`
Data source: `trpc.recovery.getWatchlist.useQuery()`

The component replaces the current Recovery KPI row. It renders as a compact 4-tile row with: Recovery Eligible (blue), Demand Outstanding (amber), Deadline Approaching (red if > 0), High Value (purple). Each tile is a clickable link to the Recovery Portal with the appropriate filter applied.

Part 4 — Phased Implementation Plan

Phase 1 — Production Defect Fixes (1.5 days)

Objective: Eliminate the two production defects that produce incorrect audit trail entries and misroute claims.

Task	File(s)	Effort
Implement `claims.closeForProcessing` procedure	`server/routers.ts`	0.5 days
Update `CloseForProcessingDialog` to use new procedure	`client/src/pages/ClaimsManagerDashboard.tsx`	0.25 days
Implement `claims.escalateClaim` procedure	`server/routers.ts`	0.5 days
Build `EscalateClaimDialog` component	`client/src/components/EscalateClaimDialog.tsx`	0.25 days
Wire Escalate button to `EscalateClaimDialog`	`client/src/pages/ClaimsManagerDashboard.tsx`	0.25 days
Write vitest tests for both new procedures	`server/*.test.ts`	0.25 days

Acceptance criteria: `closeForProcessing` creates `claim_closed` audit entry and transitions to `closed` state. `escalateClaim` creates `claim_escalated` audit entry and transitions to `manual_review` or `disputed`. No claim can be escalated via the send-back dialog.

Phase 2 — Operational Command Centre (5.5 days)

Objective: Transform the portal from a claims list viewer into an operational command centre. All six core operational questions are answerable within ten seconds of opening the portal.

Task	File(s)	Effort
Implement `claims.getQueueHealthMatrix` procedure	`server/routers.ts`	0.75 days
Implement `claims.getAttentionRequired` procedure	`server/routers.ts`	0.75 days
Implement `claims.getApprovalWorkbenchMetrics` procedure	`server/routers.ts`	0.5 days
Implement `claims.getCapacityForecast` procedure	`server/routers.ts`	0.5 days
Build `QueueHealthMatrix` component (Row 1)	`client/src/components/QueueHealthMatrix.tsx`	0.75 days
Build `AttentionRequiredWidget` component (Row 2 left)	`client/src/components/AttentionRequiredWidget.tsx`	0.5 days
Build `EscalationCentre` component (Row 2 right)	`client/src/components/EscalationCentre.tsx`	0.5 days
Build `ApprovalWorkbench` component (Row 3 left)	`client/src/components/ApprovalWorkbench.tsx`	0.5 days
Build `CapacityForecast` component (Row 3 right)	`client/src/components/CapacityForecast.tsx`	0.25 days
Demote KPI cards to compact Row 6 strip	`client/src/pages/ClaimsManagerDashboard.tsx`	0.25 days
Add Fleet Approvals to sidebar navigation	`DashboardLayout.tsx` or sidebar config	0.25 days
Integrate all new components into dashboard layout	`client/src/pages/ClaimsManagerDashboard.tsx`	0.5 days
Write vitest tests for all four new procedures	`server/*.test.ts`	0.5 days

Acceptance criteria: Dashboard opens with Queue Health Matrix as the first visible element. Attention Required widget shows a total count with breakdown. Approval Workbench shows current approval queue depth. Capacity Forecasting shows trajectory direction. Fleet Approvals is accessible from sidebar.

Phase 3 — Management Intelligence (5.5 days)

Objective: Add workforce intelligence, rework analytics, recovery watchlist, operational fraud queue, and report integration.

Task	File(s)	Effort
Implement `workflowAnalytics.getSendBackAnalytics` procedure	`server/routers/workflow-analytics.ts`	0.75 days
Implement `recovery.getWatchlist` procedure	`server/routers.ts`	0.75 days
Build `WorkforceIntelligence` component (Row 4)	`client/src/components/WorkforceIntelligence.tsx`	1.5 days
Build `RecoveryWatchlist` component	`client/src/components/RecoveryWatchlist.tsx`	0.5 days
Replace Recovery KPI row with `RecoveryWatchlist`	`client/src/pages/ClaimsManagerDashboard.tsx`	0.25 days
Add Operational Fraud Queue tiles to Fraud Alerts tab	`client/src/pages/ClaimsManagerDashboard.tsx`	0.5 days
Add per-claim report buttons to Review Queue tab	`client/src/pages/ClaimsManagerDashboard.tsx`	0.75 days
Add section report button to Fraud Alerts tab	`client/src/pages/ClaimsManagerDashboard.tsx`	0.25 days
Add structured `sendBackReason` enum to send-back dialog	`client/src/pages/ClaimsManagerDashboard.tsx` + `server/routers.ts`	0.5 days
Write vitest tests for new procedures	`server/*.test.ts`	0.5 days

Acceptance criteria: Workforce Intelligence section is visible and collapsible. Recovery Watchlist shows four actionable categories. Operational Fraud Queue tiles visible in Fraud Alerts tab. Three report buttons accessible per claim in Review Queue. Send-back dialog captures structured reason.

Phase 4 — Refinements (2.25 days)

Objective: Complete the remaining refinements identified in the audit cycle.

Task	File(s)	Effort
Add “Reopen” action to Processed Claims tab	`client/src/pages/ClaimsManagerDashboard.tsx` + `server/routers.ts`	0.5 days
Record automation threshold in `workflow_audit_trail.metadata`	`server/routers.ts` (approveClaim procedure)	0.25 days
Validate `targetRole` against `WORKFLOW_TRANSITIONS` in send-back	`server/routers.ts` (sendBackClaim procedure)	0.5 days
Merge Recently Closed card into Processed Claims tab	`client/src/pages/ClaimsManagerDashboard.tsx`	0.25 days
Add KPI trend sparklines to compact KPI strip	`client/src/pages/ClaimsManagerDashboard.tsx`	0.5 days
Final QA pass and vitest coverage review	All	0.25 days

Acceptance criteria: Closed claims can be reopened from Processed Claims tab. Automation threshold recorded in audit metadata. Send-back with invalid `targetRole` returns a clear governance error message. Recently Closed card removed; content merged into Processed tab.

Part 5 — Data Flow Summary

The following table maps every new UI component to its data source, confirming that no component requires data that does not exist in the current backend.

Component	tRPC Procedure	Backend Status	Schema Change Required
`QueueHealthMatrix`	`claims.getQueueHealthMatrix`	New procedure	No
`AttentionRequiredWidget`	`claims.getAttentionRequired`	New procedure	No
`EscalationCentre`	`claims.getEscalations` + `getAttentionRequired`	`getEscalations` exists	No
`ApprovalWorkbench`	`claims.getApprovalWorkbenchMetrics`	New procedure	No
`CapacityForecast`	`claims.getCapacityForecast`	New procedure	No
`WorkforceIntelligence` — Processors	`workflowAnalytics.getUserProductivity`	Exists	No
`WorkforceIntelligence` — Assessors	`analytics.getAssessorPerformance`	Exists	No
`WorkforceIntelligence` — Rework	`workflowAnalytics.getSendBackAnalytics`	New procedure	No
`RecoveryWatchlist`	`recovery.getWatchlist`	New procedure	No
Operational Fraud Queue	`claims.getFraudAlerts` (filtered)	Exists	No
Review Queue Report Buttons	Existing report generators	Exists	No
`closeForProcessing` fix	`claims.closeForProcessing`	New procedure	No
`escalateClaim` fix	`claims.escalateClaim`	New procedure	No
Compact KPI Strip	`claims.getManagerOverview`	Exists	No

Zero schema changes required across all four phases.

Part 6 — Total Engineering Estimate

Phase	Items	Effort	Cumulative
Phase 1 — Production Defect Fixes	6 tasks	1.5 days	1.5 days
Phase 2 — Operational Command Centre	13 tasks	5.5 days	7 days
Phase 3 — Management Intelligence	10 tasks	5.5 days	12.5 days
Phase 4 — Refinements	6 tasks	2.25 days	14.75 days
Total	**35 tasks**	**14.75 days**	

At the end of Phase 2 (7 days from start), the portal will function as a minimum viable Claims Operations Command Centre. At the end of Phase 4 (14.75 days from start), the portal will be fully aligned with the v4.0 target state architecture.

Part 7 — Acceptance Criteria Summary

The following criteria define production readiness for each phase and for the complete implementation.

Phase 1 Complete when:

- `closeForProcessing` creates `claim_closed` audit entries (not `claim_approved`)
- `escalateClaim` creates `claim_escalated` audit entries and transitions to `manual_review` / `disputed`
- No claim can reach the send-back dialog via the Escalate button

Phase 2 Complete when:

- Queue Health Matrix is the first visible element on dashboard load
- Attention Required widget shows a non-zero total when exceptions exist
- Approval Workbench shows correct counts matching the Review Queue tab
- Capacity Forecasting shows correct trajectory direction
- Fleet Approvals accessible from sidebar without navigating to the dashboard first

Phase 3 Complete when:

- Workforce Intelligence section renders with real assessor and processor data
- Recovery Watchlist shows four categories (not generic counts)
- Three report buttons accessible per claim in Review Queue
- Operational Fraud Queue tiles visible in Fraud Alerts tab

Phase 4 Complete when:

- Closed claims can be reopened from Processed Claims tab
- Send-back with invalid target role returns governance error message
- Recently Closed card is removed from the dashboard

Full Implementation Complete when:

- All 35 tasks marked complete
- All vitest tests passing
- No new TypeScript errors introduced
- Claims Manager can answer all six core operational questions within 10 seconds of opening the portal

---

End of Document — KINGA Claims Manager Portal Realignment and Redesign Document v1.0